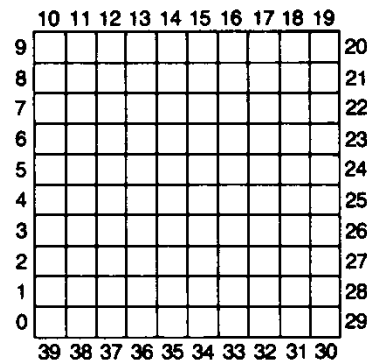


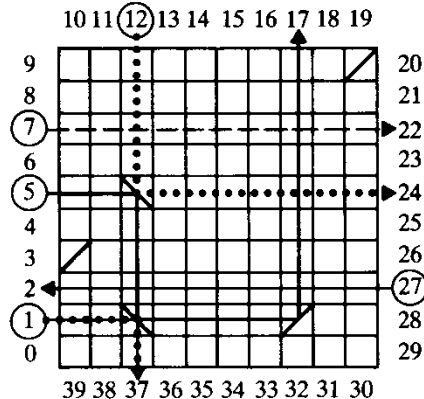
COSC120– Project # 2

Baffle Game

The game consists of a two-dimensional black box with numbers between 0 and 39 on all sides.



There are a number of obstructions, called baffles, which you cannot see, placed in the box. The object of the game is to find the baffles. You select a number between 0 and 39, which activates a laser beam originating at that location. You are then told where the beam leaves the box. If the beam does not encounter a baffle, it will exit directly opposite to where it entered. If the beam encounters a baffle, it will be deflected at right angles, either right or left, depending on the direction of the baffle. You can locate the baffles by shooting beams into the box, using the deflections of the beams as hints to the placement and direction of the baffles.



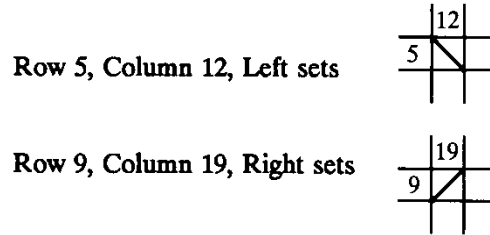
Given the box depicted above, a beam shot from 7 comes out at 22. A beam shot from 1 is deflected once and exits at 37. A beam shot from 27 exits at 2 without being deflected. A beam shot from 5 comes out at 17 after three deflections. A beam shot from 12 is deflected once, exiting at 24. The game is scored by giving one point for each laser shot and two points for each guess. A lower score, obviously, is more desirable.

This clearly ought to be an interactive program, where the baffles are set by a random number generator and a player either fires a laser beam or makes a guess as to the position of a baffle. The game has three levels: beginner, intermediate and advanced. For beginner level, there are 4 baffles. For intermediate level, there are 7 baffles. For advanced level, there are 10 baffles. Note that your program is not supposed to solve the baffles problem; it is supposed to present the game to be played.

Setting the Baffles

The baffles will be set by a random number generator. For each baffle, you will need two numbers to represent the coordinates on the grid. You can use a third random number to determine the direction of the baffle (even number = Left, odd number = Right).

Example:



Note, that you are not assured that each baffle position generated will be unique. You can only set a baffle in a "free" position—i.e., one that has not been previously set.

Input/Output

When your program first starts, you should display a welcome message and let user to choose the game level. Then you will need to display a menu with options that users can choose from and prompt the user to enter a command.

The commands are described below:

- L Laser Shot: Prompt user to input the location to shoot from (an integer from 0 through 39). Shoot the laser, with the beam entering the box at the designated location. Output: "Laser Shot #__ exited the box at __."
- G Guess: Guess the location of one baffle. Prompt the user to input the row and column (integers) and direction ('L' or 'R') of the baffle. Output: "This is guess number __." If guess is correct, print "Congratulations, you have now found __ baffles." If guess is correct but this baffle was found on a previous guess, print: "You have already found this baffle." If guess is incorrect, print: "Sorry, better luck next time."
- S Score: Print the number of shots and number of guesses made, and the current score.
- P Print: Print the box, showing the locations and directions ('R' or 'L') of all the baffles that have already been found.
- Q Quit: Stop without completing the game. Continue processing until all five baffles are found, or until the player asks to quit. On finding the last baffle, output a message of congratulations and the calculated score. Print the box showing the location and direction of all the baffles.
- C Print: Print the box, showing the locations and directions ('R' or 'L') of all the baffles but allow users to continue the game.

Sample Game

Baffles are set at the following locations:

Baffles are set at the following locations:			10	11	12	13	14	15	16	17	18	19	
Row	Column	Direction	9	-	+	-	+	-	+	-	+	-	R- 20
5	12	Left	8	-	+	-	+	-	+	-	+	-	21
9	19	Right	7	-	+	-	+	-	+	-	+	-	22
3	10	Right	6	-	+	-	+	-	+	-	+	-	23
1	12	Left	5	-	+	-	+	-	+	-	+	-	24
1	17	Right											

Player can issue the following commands:

Command Response

L 8 Laser shot #1 exited the box at 21.

L 12 Laser shot #2 exited the box at 24.

G 5 12 L This is guess #1. Congratulations, you have now found 1 baffle(s).

L 30 Laser shot #3 exited the box at 20.

G 30 20 R *** Guess out of bounds - Try again ***

G 9 19 R This is guess #2. Congratulations, you have now found 2 baffle(s).

K *** Illegal command - Try again ***

L 17 Laser shot #4 exited the box at 5.

G 5 12 L This is guess #3. You have already found this baffle.

S Number of shots : 4
 Number of guesses : 3
 Current score : 10

.

G 1 10 R This is guess #15. Sorry, better luck next time.

L 1 Laser shot #26 exited the box at 37.

G 1 12 L This is guess #16. Congratulations, you have now found 5 baffle(s).

Number of shots : 26
 Number of guesses : 16
 Current score : 58

Note, your output may differ in minor details from this example (different error messages, slight format differences, etc.).

Other requirements

Part 1:

1. Describe any variables or arrays you need to use in your program
2. Describe the functions you use for the program. Make sure to describe return type and parameters.
3. Describe your initial plan to tackle this program
4. Describe two times in which you encountered a problem and what you did to fix it.

Part 2:

Code:

1. NO literals used in your code – define constants instead.
2. Make sure that you give proper names to your variables, program files.
3. Make sure that your program is nicely indented and has meaningful header and inline comments.
4. Make sure to validate user input: laser number should be 0-30, menu options should be validated, guess input should be validated (use 0-9 or 20-29 for row and 10-19, 30-39 for column, 'L' for left (\) baffle and 'R' for right (/) baffle. If a user enters an invalid value, your program should allow them to enter another one until it is valid.
5. Use the following constants and enums

```
const int MAXCOLUMN = 10;
const int MAXROW = 10;
enum Direction {LEFT, RIGHT, UP, DOWN};
- RIGHT represents laser travels to right (column+1, row remains the same)
- LEFT represents laser travels to left (column-1, row remains the same)
- UP represents laser travels upward (column remains the same, row - 1)
- DOWN represents laser travels downward (column remains the same, row + 1)
enum Level {BEGIN=4, INTERMEDIATE=7, ADVANCED=10};
enum Grid {LEFTBAFFLE, RIGHTBAFFLE, EMPTY, LEFTFOUND, RIGHTFOUND};
- EMPTY represents no baffle
- RIGHTBAFFLE represents right (/) baffle, RIGHTFOUND represents found right (/) baffle
- LEFTBAFFLE represents left (\) baffle, LEFTFOUND represents found left (\) baffle
```

6. Use the following 2d array to keep track of game board:

```
Grid board[MAXROW][MAXCOLUMN]
```

7. You must use functions in this project. In particular, you MUST implement the following required functions:

```
- void PrintBoard(Grid board[ ][MAXCOLUMN], bool foundOnly);
This function will display the board. If foundOnly is true, display only the found baffles, else display all baffles.
```

```
- void PrintScore (int numLaserShot, int numGuess);
This function will display score information in a nice format such as below:
    Number of shots:    26
    Number of guesses:  16
    Current score:      58
```

- void SetBoard(Grid board[][MAXCOLUMN], Level gameLevel);

This function will set the baffles in the board according to the game level. Make sure to check that your don't have two baffles in the same location.

- int TrackLaser(Grid board[][MAXCOLUMN], int laserX, int laserY, Direction dir);

This function will return the exit point of a laser (0-39). LaserX and laserY should be the current location of the laser and dir is the way in which the laser is heading. This function should be a recursive function to get the end exit location returned

- bool MakeGuess (Grid board[][MAXCOLUMN], int guessR, int guessC, Direction dir);

This function will check to see if it is a correct guess or not and return true or false. It also update the board if the a correct guess is made. guessR and guessC is the laser number on the left (or right) and upper (or lower) border on the board so you need to convert them into appropriated row and column number.

8. The game should continue until the player wins or quits. Make sure that all input/output commands as described above are functioning.

Part 3:

Test the code:

Be sure to thoroughly test your code through game play. You should submit at least two complete outputs playing the game. Make sure to highlight input validation as well as all possible user choices as listed in the input/output section. Make sure to show the game being won as well as the user quitting the game.

What to hand in

A single word document with Parts 1 and 3 together.

A link to your replit project for part 2.